

```
newX = model.transform(newdata)
results.predict(newX)
```

```
Out[21]: 0    0.4791
         1    0.4961
         dtype: float64
```

4.7.3 Linear Discriminant Analysis

We begin by performing LDA on the `Smarket` data, using the function `LinearDiscriminantAnalysis()`, which we have abbreviated `LDA()`. We fit the model using only the observations before 2005.

Linear
Discriminant
Analysis()

```
In [22]: lda = LDA(store_covariance=True)
```

Since the `LDA` estimator automatically adds an intercept, we should remove the column corresponding to the intercept in both `X_train` and `X_test`. We can also directly use the labels rather than the Boolean vectors `y_train`.

```
In [23]: X_train, X_test = [M.drop(columns=['intercept'])
                           for M in [X_train, X_test]]
         lda.fit(X_train, L_train)
```

```
Out[23]: LinearDiscriminantAnalysis(store_covariance=True)
```

Here we have used the list comprehensions introduced in Section 3.6.4. Looking at our first line above, we see that the right-hand side is a list of length two. This is because the code `for M in [X_train, X_test]` iterates over a list of length two. While here we loop over a list, the list comprehension method works when looping over any iterable object. We then apply the `drop()` method to each element in the iteration, collecting the result in a list. The left-hand side tells `Python` to unpack this list of length two, assigning its elements to the variables `X_train` and `X_test`. Of course, this overwrites the previous values of `X_train` and `X_test`.

.drop()

Having fit the model, we can extract the means in the two classes with the `means_` attribute. These are the average of each predictor within each class, and are used by LDA as estimates of μ_k . These suggest that there is a tendency for the previous 2 days' returns to be negative on days when the market increases, and a tendency for the previous days' returns to be positive on days when the market declines.

```
In [24]: lda.means_
```

```
Out[24]: array([[ 0.04,  0.03],
                [-0.04, -0.03]])
```

The estimated prior probabilities are stored in the `priors_` attribute. The package `sklearn` typically uses this trailing `_` to denote a quantity estimated when using the `fit()` method. We can be sure of which entry corresponds to which label by looking at the `classes_` attribute.

```
In [25]: lda.classes_
```

```
Out[25]: array(['Down', 'Up'], dtype='<U4')
```

The LDA output indicates that $\hat{\pi}_{\text{Down}} = 0.492$ and $\hat{\pi}_{\text{Up}} = 0.508$.

```
In [26]: lda.priors_
```

```
Out[26]: array([0.492, 0.508])
```

The linear discriminant vectors can be found in the `scalings_` attribute:

```
In [27]: lda.scalings_
```

```
Out[27]: array([[ -0.642],
               [ -0.513]])
```

These values provide the linear combination of `Lag1` and `Lag2` that are used to form the LDA decision rule. In other words, these are the multipliers of the elements of $X = x$ in (4.24). If $-0.64 \times \text{Lag1} - 0.51 \times \text{Lag2}$ is large, then the LDA classifier will predict a market increase, and if it is small, then the LDA classifier will predict a market decline.

```
In [28]: lda_pred = lda.predict(X_test)
```

As we observed in our comparison of classification methods (Section 4.5), the LDA and logistic regression predictions are almost identical.

```
In [29]: confusion_table(lda_pred, L_test)
```

```
Out[29]:      Truth   Down   Up
Predicted
        Down    35    35
         Up    76   106
```

We can also estimate the probability of each class for each point in a training set. Applying a 50% threshold to the posterior probabilities of being in class one allows us to recreate the predictions contained in `lda_pred`.

```
In [30]: lda_prob = lda.predict_proba(X_test)
np.all(
    np.where(lda_prob[:,1] >= 0.5, 'Up', 'Down') == lda_pred
)
```

```
Out[30]: True
```

Above, we used the `np.where()` function that creates an array with value 'Up' for indices where the second column of `lda_prob` (the estimated posterior probability of 'Up') is greater than 0.5. For problems with more than two classes the labels are chosen as the class whose posterior probability is highest. `np.where()`

```
In [31]: np.all(
    [lda.classes_[i] for i in np.argmax(lda_prob, 1)] ==
    lda_pred
)
```

```
Out[31]: True
```

If we wanted to use a posterior probability threshold other than 50% in order to make predictions, then we could easily do so. For instance, suppose that we wish to predict a market decrease only if we are very certain that the

market will indeed decrease on that day — say, if the posterior probability is at least 90%. We know that the first column of `lda_prob` corresponds to the label `Down` after having checked the `classes_` attribute, hence we use the column index 0 rather than 1 as we did above.

```
In [32]: np.sum(lda_prob[:,0] > 0.9)
```

```
Out [32]: 0
```

No days in 2005 meet that threshold! In fact, the greatest posterior probability of decrease in all of 2005 was 52.02%.

The LDA classifier above is the first classifier from the `sklearn` library. We will use several other objects from this library. The objects follow a common structure that simplifies tasks such as cross-validation, which we will see in Chapter 5. Specifically, the methods first create a generic classifier without referring to any data. This classifier is then fit to data with the `fit()` method and predictions are always produced with the `predict()` method. This pattern of first instantiating the classifier, followed by fitting it, and then producing predictions is an explicit design choice of `sklearn`. This uniformity makes it possible to cleanly copy the classifier so that it can be fit on different data; e.g. different training sets arising in cross-validation. This standard pattern also allows for a predictable formation of workflows.

4.7.4 Quadratic Discriminant Analysis

We will now fit a QDA model to the `Smarket` data. QDA is implemented via `QuadraticDiscriminantAnalysis()` in the `sklearn` package, which we abbreviate to `QDA()`. The syntax is very similar to `LDA()`.

Quadratic
Discriminant
Analysis()

```
In [33]: qda = QDA(store_covariance=True)
         qda.fit(X_train, L_train)
```

```
Out [33]: QuadraticDiscriminantAnalysis(store_covariance=True)
```

The `QDA()` function will again compute `means_` and `priors_`.

```
In [34]: qda.means_, qda.priors_
```

```
Out [34]: (array([[ 0.04279022,  0.03389409],
                  [-0.03954635, -0.03132544]]),
          array([0.49198397, 0.50801603]))
```

The `QDA()` classifier will estimate one covariance per class. Here is the estimated covariance in the first class:

```
In [35]: qda.covariance_[0]
```

```
Out [35]: array([[ 1.50662277, -0.03924806],
                  [-0.03924806,  1.53559498]])
```

The output contains the group means. But it does not contain the coefficients of the linear discriminants, because the QDA classifier involves a quadratic, rather than a linear, function of the predictors. The `predict()` function works in exactly the same fashion as for LDA.

```
In [36]: qda_pred = qda.predict(X_test)
         confusion_table(qda_pred, L_test)
```

```
Out[36]:      Truth    Down    Up
         Predicted
           Down     30     20
           Up      81    121
```

Interestingly, the QDA predictions are accurate almost 60% of the time, even though the 2005 data was not used to fit the model.

```
In [37]: np.mean(qda_pred == L_test)
```

```
Out[37]: 0.599
```

This level of accuracy is quite impressive for stock market data, which is known to be quite hard to model accurately. This suggests that the quadratic form assumed by QDA may capture the true relationship more accurately than the linear forms assumed by LDA and logistic regression. However, we recommend evaluating this method's performance on a larger test set before betting that this approach will consistently beat the market!

4.7.5 Naive Bayes

Next, we fit a naive Bayes model to the `Smarket` data. The syntax is similar to that of `LDA()` and `QDA()`. By default, this implementation `GaussianNB()` of the naive Bayes classifier models each quantitative feature using a Gaussian distribution. However, a kernel density method can also be used to estimate the distributions.

`GaussianNB()`

```
In [38]: NB = GaussianNB()
         NB.fit(X_train, L_train)
```

```
Out[38]: GaussianNB()
```

The classes are stored as `classes_`.

```
In [39]: NB.classes_
```

```
Out[39]: array(['Down', 'Up'], dtype='<U4')
```

The class prior probabilities are stored in the `class_prior_` attribute.

```
In [40]: NB.class_prior_
```

```
Out[40]: array([0.49, 0.51])
```

The parameters of the features can be found in the `theta_` and `var_` attributes. The number of rows is equal to the number of classes, while the number of columns is equal to the number of features. We see below that the mean for feature `Lag1` in the `Down` class is 0.043.

```
In [41]: NB.theta_
```

```
Out[41]: array([[ 0.043,  0.034],
                [-0.040, -0.031]])
```

Its variance is 1.503.

```
In [42]: NB.var_
```

```
Out[42]: array([[1.503, 1.532],
               [1.514, 1.487]])
```

How do we know the names of these attributes? We use `NB?` (or `?NB`).

We can easily verify the mean computation:

```
In [43]: X_train[L_train == 'Down'].mean()
```

```
Out[43]: Lag1      0.042790
         Lag2      0.033894
         dtype: float64
```

Similarly for the variance:

```
In [44]: X_train[L_train == 'Down'].var(ddof=0)
```

```
Out[44]: Lag1      1.503554
         Lag2      1.532467
         dtype: float64
```

The `GaussianNB()` function calculates variances using the $1/n$ formula.⁶ Since `NB()` is a classifier in the `sklearn` library, making predictions uses the same syntax as for `LDA()` and `QDA()` above.

```
In [45]: nb_labels = NB.predict(X_test)
         confusion_table(nb_labels, L_test)
```

```
Out[45]:   Truth   Down   Up
         Predicted
         Down    29    20
         Up     82   121
```

Naive Bayes performs well on these data, with accurate predictions over 59% of the time. This is slightly worse than QDA, but much better than LDA.

As for LDA, the `predict_proba()` method estimates the probability that each observation belongs to a particular class.

```
In [46]: NB.predict_proba(X_test)[:5]
```

```
Out[46]: array([[0.4873, 0.5127],
               [0.4762, 0.5238],
               [0.4653, 0.5347],
               [0.4748, 0.5252],
               [0.4902, 0.5098]])
```

4.7.6 K-Nearest Neighbors

We will now perform KNN using the `KNeighborsClassifier()` function. This

`KNeighborsClassifier()`

⁶There are two formulas for computing the sample variance of n observations $x_1, \dots, x_n$: $\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$ and $\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$ where $\bar{x}$ is the sample mean. In most cases the distinction is not important.

function works similarly to the other model-fitting functions that we have encountered thus far.

As is the case for LDA and QDA, we fit the classifier using the `fit` method. New predictions are formed using the `predict` method of the object returned by `fit()`.

```
In [47]: knn1 = KNeighborsClassifier(n_neighbors=1)
knn1.fit(X_train, L_train)
knn1_pred = knn1.predict(X_test)
confusion_table(knn1_pred, L_test)
```

```
Out[47]:      Truth    Down  Up
Predicted
      Down     43   58
      Up      68   83
```

The results using $K = 1$ are not very good, since only 50% of the observations are correctly predicted. Of course, it may be that $K = 1$ results in an overly-flexible fit to the data.

```
In [48]: (83+43)/252, np.mean(knn1_pred == L_test)
```

```
Out[48]: (0.5, 0.5)
```

We repeat the analysis below using $K = 3$.

```
In [49]: knn3 = KNeighborsClassifier(n_neighbors=3)
knn3_pred = knn3.fit(X_train, L_train).predict(X_test)
np.mean(knn3_pred == L_test)
```

```
Out[49]: 0.532
```

The results have improved slightly. But increasing K further provides no further improvements. It appears that for these data, and this train/test split, QDA gives the best results of the methods that we have examined so far.

KNN does not perform well on the `Smarket` data, but it often does provide impressive results. As an example we will apply the KNN approach to the `Caravan` data set, which is part of the `ISLP` library. This data set includes 85 predictors that measure demographic characteristics for 5,822 individuals. The response variable is `Purchase`, which indicates whether or not a given individual purchases a caravan insurance policy. In this data set, only 6% of people purchased caravan insurance.

```
In [50]: Caravan = load_data('Caravan')
Purchase = Caravan.Purchase
Purchase.value_counts()
```

```
Out[50]: No      5474
Yes       348
Name: Purchase, dtype: int64
```

The method `value_counts()` takes a `pd.Series` or `pd.DataFrame` and returns a `pd.Series` with the corresponding counts for each unique element. In this case `Purchase` has only `Yes` and `No` values and returns how many values of each there are.

```
In [51]: 348 / 5822
```

```
Out [51]: 0.0598
```

Our features will include all columns except `Purchase`.

```
In [52]: feature_df = Caravan.drop(columns=['Purchase'])
```

Because the KNN classifier predicts the class of a given test observation by identifying the observations that are nearest to it, the scale of the variables matters. Any variables that are on a large scale will have a much larger effect on the *distance* between the observations, and hence on the KNN classifier, than variables that are on a small scale. For instance, imagine a data set that contains two variables, `salary` and `age` (measured in dollars and years, respectively). As far as KNN is concerned, a difference of 1,000 USD in salary is enormous compared to a difference of 50 years in age. Consequently, `salary` will drive the KNN classification results, and `age` will have almost no effect. This is contrary to our intuition that a salary difference of 1,000 USD is quite small compared to an age difference of 50 years. Furthermore, the importance of scale to the KNN classifier leads to another issue: if we measured `salary` in Japanese yen, or if we measured `age` in minutes, then we'd get quite different classification results from what we get if these two variables are measured in dollars and years.

A good way to handle this problem is to *standardize* the data so that all variables are given a mean of zero and a standard deviation of one. Then all variables will be on a comparable scale. This is accomplished using the `StandardScaler()` transformation.

standardize

```
In [53]: scaler = StandardScaler(with_mean=True,
                                with_std=True,
                                copy=True)
```

Standard
Scaler()

The argument `with_mean` indicates whether or not we should subtract the mean, while `with_std` indicates whether or not we should scale the columns to have standard deviation of 1 or not. Finally, the argument `copy=True` indicates that we will always copy data, rather than trying to do calculations in place where possible.

This transformation can be fit and then applied to arbitrary data. In the first line below, the parameters for the scaling are computed and stored in `scaler`, while the second line actually constructs the standardized set of features.

```
In [54]: scaler.fit(feature_df)
X_std = scaler.transform(feature_df)
```

Now every column of `feature_std` below has a standard deviation of one and a mean of zero.

```
In [55]: feature_std = pd.DataFrame(
            X_std,
            columns=feature_df.columns);
feature_std.std()
```

```
Out [55]: MOSTYPE      1.000086
          MAANTHUI     1.000086
```

```

MGEMOMV      1.000086
MGEMLEEF      1.000086
MOSHOOFD      1.000086
...
AZEILPL       1.000086
APLEZIER      1.000086
AFIETS        1.000086
AINBOED       1.000086
ABYSTAND      1.000086
Length: 85, dtype: float64

```

Notice that the standard deviations are not quite 1 here; this is again due to some procedures using the $1/n$ convention for variances (in this case `scaler()`), while others use $1/(n-1)$ (the `std()` method). See the footnote on page 183. In this case it does not matter, as long as the variables are all on the same scale.

Using the function `train_test_split()` we now split the observations into a test set, containing 1000 observations, and a training set containing the remaining observations. The argument `random_state=0` ensures that we get the same split each time we rerun the code.

```

In [56]: (X_train,
          X_test,
          y_train,
          y_test) = train_test_split(feature_std,
                                     Purchase,
                                     test_size=1000,
                                     random_state=0)

```

`?train_test_split` reveals that the non-keyword arguments can be `lists`, `arrays`, `pandas dataframes` etc that all have the same length (`shape[0]`) and hence are *indexable*. In this case they are the dataframe `feature_std` and the response variable `Purchase`. We fit a KNN model on the training data using $K = 1$, and evaluate its performance on the test data.

```

In [57]: knn1 = KNeighborsClassifier(n_neighbors=1)
knn1_pred = knn1.fit(X_train, y_train).predict(X_test)
np.mean(y_test != knn1_pred), np.mean(y_test != "No")

```

```
Out[57]: (0.111, 0.067)
```

The KNN error rate on the 1,000 test observations is about 11%. At first glance, this may appear to be fairly good. However, since just over 6% of customers purchased insurance, we could get the error rate down to almost 6% by always predicting `No` regardless of the values of the predictors! This is known as the *null rate*.

Suppose that there is some non-trivial cost to trying to sell insurance to a given individual. For instance, perhaps a salesperson must visit each potential customer. If the company tries to sell insurance to a random selection of customers, then the success rate will be only 6%, which may be far too low given the costs involved. Instead, the company would like to try to sell insurance only to customers who are likely to buy it. So the overall error rate is not of interest. Instead, the fraction of individuals that are correctly predicted to buy insurance is of interest.

```

In [58]: confusion_table(knn1_pred, y_test)

```

```
Out[58]:      Truth   No   Yes
      Predicted
         No   880   58
         Yes   53    9
```

It turns out that KNN with $K = 1$ does far better than random guessing among the customers that are predicted to buy insurance. Among 62 such customers, 9, or 14.5%, actually do purchase insurance. This is double the rate that one would obtain from random guessing.

```
In [59]: 9/(53+9)
```

```
Out[59]: 0.145
```

Tuning Parameters

The number of neighbors in KNN is referred to as a *tuning parameter*, also referred to as a *hyperparameter*. We do not know *a priori* what value to use. It is therefore of interest to see how the classifier performs on test data as we vary these parameters. This can be achieved with a **for** loop, described in Section 2.3.8. Here we use a for loop to look at the accuracy of our classifier in the group predicted to purchase insurance as we vary the number of neighbors from 1 to 5:

tuning
parameter
hyper-
parameter

```
In [60]: for K in range(1,6):
          knn = KNeighborsClassifier(n_neighbors=K)
          knn_pred = knn.fit(X_train, y_train).predict(X_test)
          C = confusion_table(knn_pred, y_test)
          templ = ('K={0:d}: # predicted to rent: {1:>2}, ' +
                  ' # who did rent {2:d}, accuracy {3:.1%} ')
          pred = C.loc['Yes'].sum()
          did_rent = C.loc['Yes', 'Yes']
          print(templ.format(
              K,
              pred,
              did_rent,
              did_rent / pred))
```

```
K=1: # predicted to rent: 62, # who did rent 9, accuracy 14.5%
K=2: # predicted to rent: 6, # who did rent 1, accuracy 16.7%
K=3: # predicted to rent: 20, # who did rent 3, accuracy 15.0%
K=4: # predicted to rent: 3, # who did rent 0, accuracy 0.0%
K=5: # predicted to rent: 7, # who did rent 1, accuracy 14.3%
```

We see some variability — the numbers for $K=4$ are very different from the rest.

Comparison to Logistic Regression

As a comparison, we can also fit a logistic regression model to the data. This can also be done with **sklearn**, though by default it fits something like the *ridge regression* version of logistic regression, which we introduce in Chapter 6. This can be modified by appropriately setting the argument **C** below. Its default value is 1 but by setting it to a very large number, the algorithm converges to the same solution as the usual (unregularized) logistic regression estimator discussed above.

Unlike the `statsmodels` package, `sklearn` focuses less on inference and more on classification. Hence, the `summary` methods seen in `statsmodels` and our simplified version seen with `summarize` are not generally available for the classifiers in `sklearn`.

```
In [61]: logit = LogisticRegression(C=1e10, solver='liblinear')
logit.fit(X_train, y_train)
logit_pred = logit.predict_proba(X_test)
logit_labels = np.where(logit_pred[:,1] > 5, 'Yes', 'No')
confusion_table(logit_labels, y_test)
```

```
Out[61]:      Truth    No  Yes
Predicted
        No   933   67
        Yes    0    0
```

We used the argument `solver='liblinear'` above to avoid a warning with the default solver which would indicate that the algorithm does not converge.

If we use 0.5 as the predicted probability cut-off for the classifier, then we have a problem: none of the test observations are predicted to purchase insurance. However, we are not required to use a cut-off of 0.5. If we instead predict a purchase any time the predicted probability of purchase exceeds 0.25, we get much better results: we predict that 29 people will purchase insurance, and we are correct for about 31% of these people. This is almost five times better than random guessing!

```
In [62]: logit_labels = np.where(logit_pred[:,1]>0.25, 'Yes', 'No')
confusion_table(logit_labels, y_test)
```

```
Out[62]:      Truth    No  Yes
Predicted
        No   913   58
        Yes    20    9
```

```
In [63]: 9/(20+9)
```

```
Out[63]: 0.310
```

4.7.7 Linear and Poisson Regression on the Bikeshare Data

Here we fit linear and Poisson regression models to the `Bikeshare` data, as described in Section 4.6. The response `bikers` measures the number of bike rentals per hour in Washington, DC in the period 2010–2012.

```
In [64]: Bike = load_data('Bikeshare')
```

Let's have a peek at the dimensions and names of the variables in this dataframe.

```
In [65]: Bike.shape, Bike.columns
```

```
Out[65]: ((8645, 15),
Index(['season', 'mnth', 'day', 'hr', 'holiday', 'weekday',
      'workingday', 'weathersit', 'temp', 'atemp', 'hum',
      'windspeed', 'casual', 'registered', 'bikers'],
      dtype='object'))
```

Linear Regression

We begin by fitting a linear regression model to the data.

```
In [66]: X = MS(['mnth',
                'hr',
                'workingday',
                'temp',
                'weathersit']).fit_transform(Bike)
Y = Bike['bikers']
M_lm = sm.OLS(Y, X).fit()
summarize(M_lm)
```

```
Out[66]:
```

	coef	std err	t	P> t
intercept	-68.6317	5.307	-12.932	0.000
mnth[Feb]	6.8452	4.287	1.597	0.110
mnth[March]	16.5514	4.301	3.848	0.000
mnth[April]	41.4249	4.972	8.331	0.000
mnth[May]	72.5571	5.641	12.862	0.000
mnth[June]	67.8187	6.544	10.364	0.000
mnth[July]	45.3245	7.081	6.401	0.000
mnth[Aug]	53.2430	6.640	8.019	0.000
mnth[Sept]	66.6783	5.925	11.254	0.000
mnth[Oct]	75.8343	4.950	15.319	0.000
mnth[Nov]	60.3100	4.610	13.083	0.000
mnth[Dec]	46.4577	4.271	10.878	0.000
hr[1]	-14.5793	5.699	-2.558	0.011
hr[2]	-21.5791	5.733	-3.764	0.000
hr[3]	-31.1408	5.778	-5.389	0.000
.....				

There are 24 levels in `hr` and 40 rows in all, so we have truncated the summary. In `M_lm`, the first levels `hr[0]` and `mnth[Jan]` are treated as the baseline values, and so no coefficient estimates are provided for them: implicitly, their coefficient estimates are zero, and all other levels are measured relative to these baselines. For example, the Feb coefficient of 6.845 signifies that, holding all other variables constant, there are on average about 7 more riders in February than in January. Similarly there are about 16.5 more riders in March than in January.

The results seen in Section 4.6.1 used a slightly different coding of the variables `hr` and `mnth`, as follows:

```
In [67]: hr_encode = contrast('hr', 'sum')
mnth_encode = contrast('mnth', 'sum')
```

Refitting again:

```
In [68]: X2 = MS([mnth_encode,
                  hr_encode,
                  'workingday',
                  'temp',
```

```
'weathersit']).fit_transform(Bike)
M2_lm = sm.OLS(Y, X2).fit()
S2 = summarize(M2_lm)
S2
```

```
Out[68]:
```

	coef	std err	t	P> t
intercept	73.5974	5.132	14.340	0.000
mnth[Jan]	-46.0871	4.085	-11.281	0.000
mnth[Feb]	-39.2419	3.539	-11.088	0.000
mnth[March]	-29.5357	3.155	-9.361	0.000
mnth[April]	-4.6622	2.741	-1.701	0.089
mnth[May]	26.4700	2.851	9.285	0.000
mnth[June]	21.7317	3.465	6.272	0.000
mnth[July]	-0.7626	3.908	-0.195	0.845
mnth[Aug]	7.1560	3.535	2.024	0.043
mnth[Sept]	20.5912	3.046	6.761	0.000
mnth[Oct]	29.7472	2.700	11.019	0.000
mnth[Nov]	14.2229	2.860	4.972	0.000
hr[0]	-96.1420	3.955	-24.307	0.000
hr[1]	-110.7213	3.966	-27.916	0.000
hr[2]	-117.7212	4.016	-29.310	0.000
.....				

What is the difference between the two codings? In `M2_lm`, a coefficient estimate is reported for all but level `23` of `hr` and level `Dec` of `mnth`. Importantly, in `M2_lm`, the (unreported) coefficient estimate for the last level of `mnth` is not zero: instead, it equals the negative of the sum of the coefficient estimates for all of the other levels. Similarly, in `M2_lm`, the coefficient estimate for the last level of `hr` is the negative of the sum of the coefficient estimates for all of the other levels. This means that the coefficients of `hr` and `mnth` in `M2_lm` will always sum to zero, and can be interpreted as the difference from the mean level. For example, the coefficient for January of -46.087 indicates that, holding all other variables constant, there are typically 46 fewer riders in January relative to the yearly average.

It is important to realize that the choice of coding really does not matter, provided that we interpret the model output correctly in light of the coding used. For example, we see that the predictions from the linear model are the same regardless of coding:

```
In[69]: np.sum((M_lm.fittedvalues - M2_lm.fittedvalues)**2)
```

```
Out[69]: 1.53e-20
```

The sum of squared differences is zero. We can also see this using the `np.allclose()` function:

```
In[70]: np.allclose(M_lm.fittedvalues, M2_lm.fittedvalues)
```

```
np.allclose()
```

```
Out[70]: True
```

To reproduce the left-hand side of Figure 4.13 we must first obtain the coefficient estimates associated with `mnth`. The coefficients for January through November can be obtained directly from the `M2_lm` object. The coefficient for December must be explicitly computed as the negative sum of all the other months. We first extract all the coefficients for month from the coefficients of `M2_lm`.

```
In [71]: coef_month = S2[S2.index.str.contains('mnth')]['coef']
coef_month
```

```
Out [71]: mnth[Jan]      -46.0871
          mnth[Feb]     -39.2419
          mnth[March]   -29.5357
          mnth[April]   -4.6622
          mnth[May]     26.4700
          mnth[June]    21.7317
          mnth[July]    -0.7626
          mnth[Aug]      7.1560
          mnth[Sept]    20.5912
          mnth[Oct]     29.7472
          mnth[Nov]     14.2229
          Name: coef, dtype: float64
```

Next, we append `Dec` as the negative of the sum of all other months.

```
In [72]: months = Bike['mnth'].dtype.categories
coef_month = pd.concat([
    coef_month,
    pd.Series([-coef_month.sum()],
              index=['mnth[Dec]'],
              dtype=coef_month.dtype),
])
coef_month
```

```
Out [72]: mnth[Jan]      -46.0871
          mnth[Feb]     -39.2419
          mnth[March]   -29.5357
          mnth[April]   -4.6622
          mnth[May]     26.4700
          mnth[June]    21.7317
          mnth[July]    -0.7626
          mnth[Aug]      7.1560
          mnth[Sept]    20.5912
          mnth[Oct]     29.7472
          mnth[Nov]     14.2229
          mnth[Dec]      0.3705
          Name: coef, dtype: float64
```

Finally, to make the plot neater, we'll just use the first letter of each month, which is the 6th entry of each of the labels in the index.

```
In [73]: fig_month, ax_month = subplots(figsize=(8,8))
x_month = np.arange(coef_month.shape[0])
ax_month.plot(x_month, coef_month, marker='o', ms=10)
ax_month.set_xticks(x_month)
ax_month.set_xticklabels([l[5] for l in coef_month.index], fontsize=20)
ax_month.set_xlabel('Month', fontsize=20)
ax_month.set_ylabel('Coefficient', fontsize=20);
```

Reproducing the right-hand plot in Figure 4.13 follows a similar process.

```
In [74]: coef_hr = S2[S2.index.str.contains('hr')]['coef']
coef_hr = coef_hr.reindex(['hr[{0}]'.format(h) for h in range(23)])
coef_hr = pd.concat([coef_hr,
```

```
pd.Series([-coef_hr.sum()], index=['hr[23]'])
])
```

We now make the hour plot.

```
In [75]: fig_hr, ax_hr = subplots(figsize=(8,8))
x_hr = np.arange(coef_hr.shape[0])
ax_hr.plot(x_hr, coef_hr, marker='o', ms=10)
ax_hr.set_xticks(x_hr[::2])
ax_hr.set_xticklabels(range(24)[::2], fontsize=20)
ax_hr.set_xlabel('Hour', fontsize=20)
ax_hr.set_ylabel('Coefficient', fontsize=20);
```

Poisson Regression

Now we fit instead a Poisson regression model to the **Bikeshare** data. Very little changes, except that we now use the function `sm.GLM()` with the Poisson family specified:

```
In [76]: M_pois = sm.GLM(Y, X2, family=sm.families.Poisson()).fit()
```

We can plot the coefficients associated with `mnth` and `hr`, in order to reproduce Figure 4.15. We first complete these coefficients as before.

```
In [77]: S_pois = summarize(M_pois)
coef_month = S_pois[S_pois.index.str.contains('mnth']]['coef']
coef_month = pd.concat([coef_month,
                        pd.Series([-coef_month.sum()],
                                index=['mnth[Dec]'])])
coef_hr = S_pois[S_pois.index.str.contains('hr']]['coef']
coef_hr = pd.concat([coef_hr,
                    pd.Series([-coef_hr.sum()],
                                index=['hr[23]'])])
```

The plotting is as before.

```
In [78]: fig_pois, (ax_month, ax_hr) = subplots(1, 2, figsize=(16,8))
ax_month.plot(x_month, coef_month, marker='o', ms=10)
ax_month.set_xticks(x_month)
ax_month.set_xticklabels([l[5] for l in coef_month.index], fontsize=20)
ax_month.set_xlabel('Month', fontsize=20)
ax_month.set_ylabel('Coefficient', fontsize=20)
ax_hr.plot(x_hr, coef_hr, marker='o', ms=10)
ax_hr.set_xticklabels(range(24)[::2], fontsize=20)
ax_hr.set_xlabel('Hour', fontsize=20)
ax_hr.set_ylabel('Coefficient', fontsize=20);
```

We compare the fitted values of the two models. The fitted values are stored in the `fittedvalues` attribute returned by the `fit()` method for both the linear regression and the Poisson fits. The linear predictors are stored as the attribute `lin_pred`.

```
In [79]: fig, ax = subplots(figsize=(8, 8))
ax.scatter(M2_lm.fittedvalues,
          M_pois.fittedvalues,
          s=20)
ax.set_xlabel('Linear Regression Fit', fontsize=20)
```